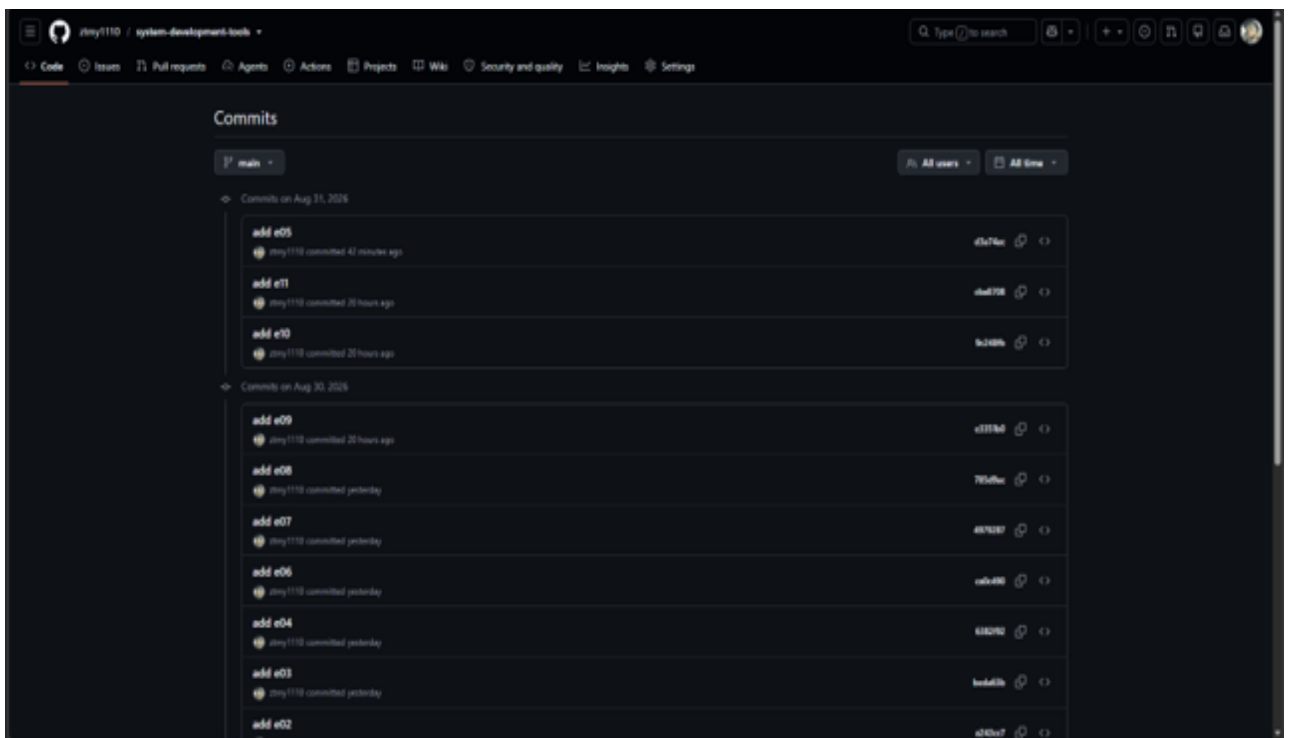


《系统开发工具基础》实验报告

姓名：夏昕 学号：25080012068
专业：软件工程 任课教师：周小伟
实验时间：2026/8/24 实验周次：第 1 周

GitHub 仓库链接：<https://github.com/ztmy1110/system-development-tools>

commit 记录



课堂习题

第 1 题：含空格文件名的批量整理

一、主题

Shell 基础与文件系统

二、题目要求

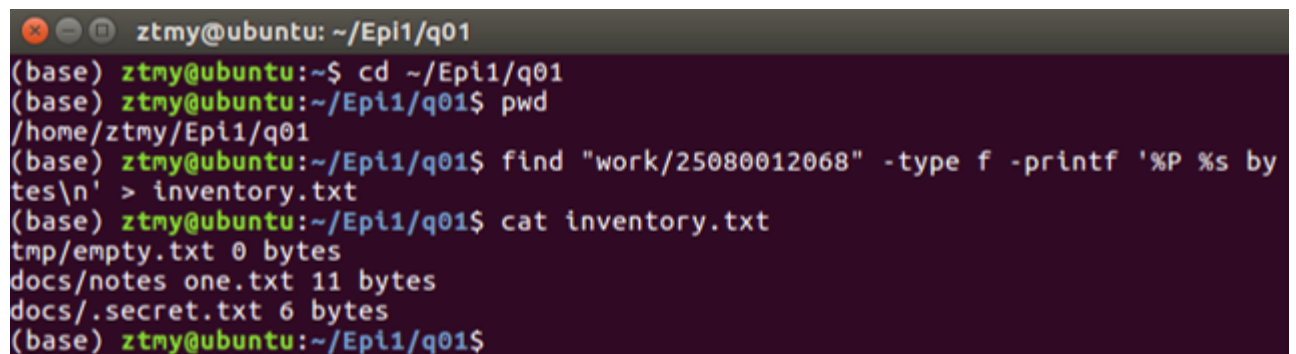
1. 创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）。
2. 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件。
3. 把 input 下所有.txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理名称中的空格。

4. 将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

三、源程序代码

```
cd ~/Epi1
mkdir -p q01
cd q01
mkdir -p input/docs input/tmp
echo -e "alpha\nbeta" > "input/docs/notes one.txt"
echo "hidden" > "input/docs/.secret.txt"
touch "input/tmp/empty.txt"
echo -e "log line 1\nlog line 2" > input/run.log
pwd
ls -laR input
mkdir -p "work/25080012068/docs"
mkdir -p "work/25080012068/tmp"
find input/docs -type f -name "*.txt" -exec cp { } "work/25080012068/docs/" \;
find input/tmp -type f -name "*.txt" -exec cp { } "work/25080012068/tmp/" \;
find "work/25080012068" -type d -exec chmod 750 { } \;
find "work/25080012068" -type f -exec chmod 640 { } \;
find "work/25080012068" -type f -printf '%P %s bytes\n' > inventory.txt
cat inventory.txt
```

四、程序运行结果



```
ztmy@ubuntu: ~/Epi1/q01
(base) ztmy@ubuntu:~$ cd ~/Epi1/q01
(base) ztmy@ubuntu:~/Epi1/q01$ pwd
/home/ztmy/Epi1/q01
(base) ztmy@ubuntu:~/Epi1/q01$ find "work/25080012068" -type f -printf '%P %s bytes\n' > inventory.txt
(base) ztmy@ubuntu:~/Epi1/q01$ cat inventory.txt
tmp/empty.txt 0 bytes
docs/notes one.txt 11 bytes
docs/.secret.txt 6 bytes
(base) ztmy@ubuntu:~/Epi1/q01$
```

五、解题感悟

1. 熟悉了 mkdir、cp、find、chmod 等常用 Shell 命令。
2. 认识到处理带空格和隐藏文件时需要注意路径和文件名的写法。
3. 对 Linux 文件权限以及命令行批量处理文件有了更直观的理解。

第 2 题：随机访问日志统计

一、主题

Shell 管道与文本处理

二、题目要求

1. 编写 analyze.sh，接收一个 CSV 文件路径；文件不存在时将错误写入 stderr 并返回非零退出码。
2. 使用 Shell 管道以及 awk、sort、head 等工具，输出 HTTP5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序。
3. 输出全部数据行的平均 latency_ms，保留两位小数；表头不得计入。
4. 分别对 access.csv 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

三、源程序代码

```
cd ~/Epi1/q02
```

```
touch access.csv analyze.sh
```

```
vim access.csv
```

```
vim analyze.sh
```

```
./analyze.sh access.csv
```

```
./analyze.sh test.csv
```

analyze.sh

```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then
```

```
echo "Usage: $0 <csv_file>" >&2
```

```
exit 1
```

```
fi
```

```
if [ ! -f "$1" ]; then
```

```
echo "Error: file '$1' does not exist" >&2
```

```
exit 1
```

```
fi
```

```
echo "Top 2 HTTP 5xx paths:"
```

```
awk -F, '
```

```
NR > 1 && $4 >= 500 && $4 < 600 {
```

```
count[$3]++
```

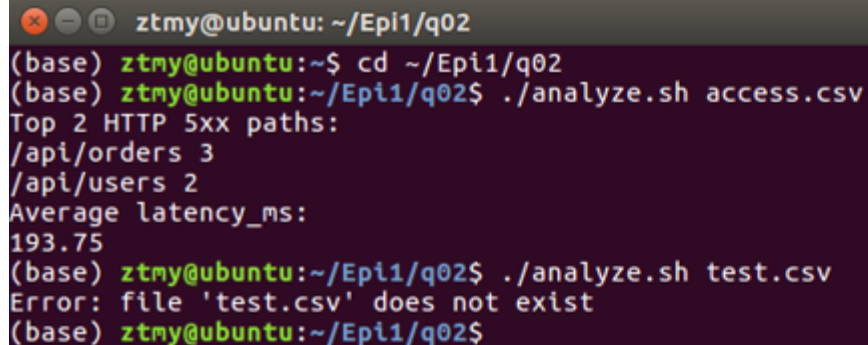
```
}
```

```

END {
for (path in count) {
print path, count[path]
}
}
' "$1" |
sort -k2,2nr -k1,1 |
head -2
echo "Average latency_ms:"
awk -F, '
NR > 1 {
sum += $5
count++
}
END {
printf "%0.2f\n", sum / count
}
' "$1"

```

四、程序运行结果



```

ztmy@ubuntu: ~/Epi1/q02
(base) ztmy@ubuntu:~$ cd ~/Epi1/q02
(base) ztmy@ubuntu:~/Epi1/q02$ ./analyze.sh access.csv
Top 2 HTTP 5xx paths:
/api/orders 3
/api/users 2
Average latency_ms:
193.75
(base) ztmy@ubuntu:~/Epi1/q02$ ./analyze.sh test.csv
Error: file 'test.csv' does not exist
(base) ztmy@ubuntu:~/Epi1/q02$

```

五、解题感悟

1. 熟悉了 awk、sort、head 以及管道的基本使用方法。
2. 学会了通过多个 Shell 命令组合完成 CSV 数据统计。

3. 加深了对 Shell 脚本、错误处理和管道处理思想的理解。

第 3 题：制造、解决并解释一次合并冲突

一、主题

Version Control and Git

二、题目要求

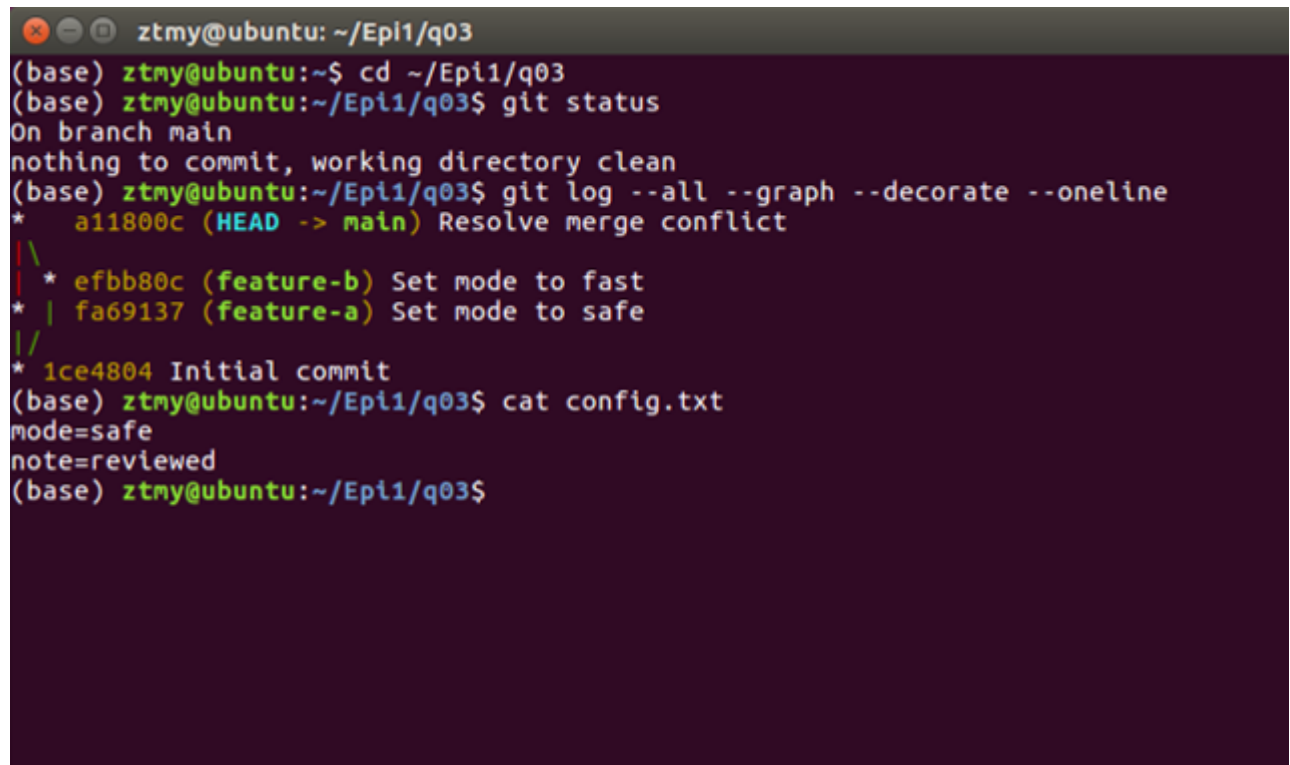
1. 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
2. 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
3. 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行
note=reviewed。
4. 确认工作区干净，运行 gitlog -all -graph -decorate --oneline 查看历史。

三、源程序代码

```
cd ~/Epi1
mkdir -p q03
cd q03
git init
echo "mode=normal" > config.txt
git add config.txt
git commit -m "Initial commit"
git branch -M main
git checkout -b feature-a
echo "mode=safe" > config.txt
git add config.txt
git commit -m "Set mode to safe"
git checkout main
git checkout -b feature-b
echo "mode=fast" > config.txt
git add config.txt
git commit -m "Set mode to fast"
git checkout main
git merge feature-a
git merge feature-b
printf "mode=safe\nnote=reviewed\n" > config.txt
git add config.txt
git commit -m "Resolve merge conflict"
git status
```

```
git log --all --graph --decorate --oneline
cat config.txt
```

四、程序运行结果

A terminal window with a dark background and light green text. The prompt is 'ztmy@ubuntu: ~/Epi1/q03'. The user enters '(base) ztmy@ubuntu:~\$ cd ~/Epi1/q03'. The prompt changes to '(base) ztmy@ubuntu:~/Epi1/q03\$'. The user enters 'git status'. The output is 'On branch main' and 'nothing to commit, working directory clean'. The user enters 'git log --all --graph --decorate --oneline'. The output shows a commit graph with three commits: 'a11800c (HEAD -> main) Resolve merge conflict', 'efbb80c (feature-b) Set mode to fast', and 'fa69137 (feature-a) Set mode to safe'. The user enters 'cat config.txt'. The output is 'mode=safe' and 'note=reviewed'. The prompt returns to '(base) ztmy@ubuntu:~/Epi1/q03\$'.

五、解题感悟

1. 进一步理解了 Git 中分支、提交和合并的基本概念。
2. 通过实际操作认识了合并冲突产生的原因以及解决方法。
3. 对 Git 的版本管理和分支协作流程有了更加清晰的认识。

第 4 题：修复并构建一页技术说明

一、主题

LaTeX 文档编辑

二、题目要求

1. 加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用。
2. 加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表。
3. 使用 latexmk-pdf-halt-on-error 或 tectonic 生成 report.pdf。
4. 确认 PDF 能打开，且交叉引用不显示“??”。

三、源程序代码

```
cd ~/Epi1
mkdir -p q04
cd q04
vim report.tex
```

```
latexmk -pdf -halt-on-error report.tex
```

```
ls -lh
```

```
xdg-open report.pdf
```

```
report.tex
```

```
\documentclass{article}
```

```
\usepackage{amsmath}
```

```
\begin{document}
```

```
\title{Tool Report}
```

```
\author{Xia Xin-25080012068}
```

```
\maketitle
```

```
\section{Result}
```

This experiment demonstrates the use of Git and LaTeX tools.

The following equation describes the relationship between energy, mass, and the speed of light:

```
\begin{equation}
```

$$E = mc^2$$

```
\label{eq:energy}
```

```
\end{equation}
```

As shown in Equation~\eqref{eq:energy}, E represents energy,

m represents mass, and c represents the speed of light in vacuum.

The experimental results are shown in Table~\ref{tab:result}.

```
\begin{table}[htbp]
```

```
\centering
```

```
\caption{Tool Experiment Results}
```

```
\label{tab:result}
```

```
\begin{tabular}{cc}
```

```
\hline
```

```
Tool & Result \\
```

```
\hline
```

```
Git & Pass \\
```

```
LaTeX & Pass \\
```

```
\hline
```

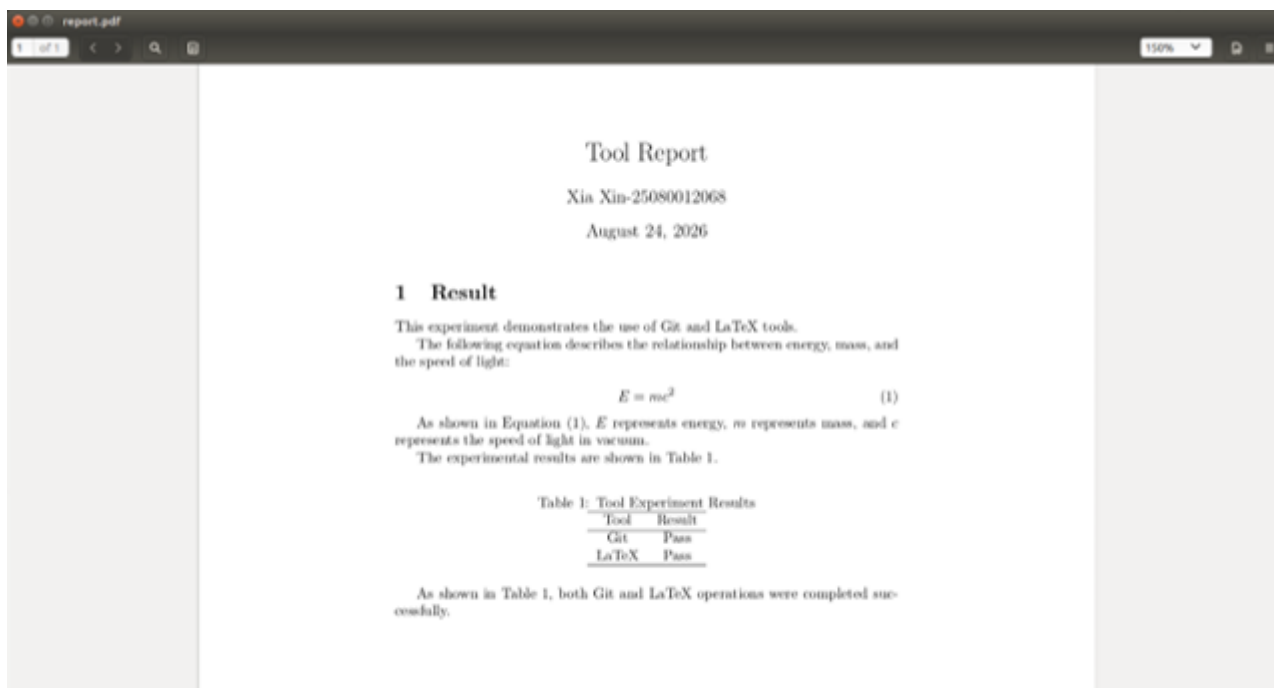
```
\end{tabular}
```

```
\end{table}
```

As shown in Table~\ref{tab:result}, both Git and LaTeX operations were completed successfully.

```
\end{document}
```

四、程序运行结果



五、解题感悟

1. 熟悉了 LaTeX 文档的基本结构以及命令行编译方法。
2. 学会了使用 label、ref 和 eqref 实现公式和表格的交叉引用。
3. 加深了对 LaTeX 技术文档排版和 PDF 生成流程的理解。

课后练习

练习 1：脚本编写与文件读取

一、主题

Shell 基础与文件系统

二、题目要求

写一个脚本，接收文件名参数（\$1），用 test -f 或 [-f ...] 检查该文件是否存在，并根据结果输出不同提示。

三、源程序代码

check.sh

```
#!/bin/bash
if [ -f "$1" ];then
echo "file exists: $1"
else
echo "file not found: $1"
fi
```

四、程序运行结果

```
ztmy@ubuntu: ~/Epi1/e01
(base) ztmy@ubuntu:~$ cd ~/Epi1/e01
(base) ztmy@ubuntu:~/Epi1/e01$ ./check.sh test.txt
file exists: test.txt
(base) ztmy@ubuntu:~/Epi1/e01$ ./check.sh test.pdf
file not found: test.pdf
(base) ztmy@ubuntu:~/Epi1/e01$
```

五、解题感悟

1. 学会了使用 '\$1' 获取脚本运行时传入的文件名参数。
2. 熟悉了 'test -f' 和 '[-f ...]' 判断文件是否存在的方法。
3. 加深了对 Shell 中条件判断和分支结构的理解。

练习 2: shell 的标准流与重定向

一、主题

Shell 基础与文件系统

二、题目要求

Shell 有三条标准流: stdin (0)、stdout (1)、stderr (2)。运行 `ls /nonexistent /tmp`, 把 stdout 和 stderr 分别重定向到两个文件。你将如何把两者都重定向到同一个文件?

三、源程序代码

```
cd Epi1/e02
touch stdout.txt stderr.txt
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
cat stdout.txt stderr.txt
touch output.txt
ls /nonexistent /tmp > output.txt 2>&1
cat output.txt
```

四、程序运行结果

```

ztmy@ubuntu: ~/Epi1/e02
(base) ztmy@ubuntu:~$ cd Epi1/e02
(base) ztmy@ubuntu:~/Epi1/e02$ touch stdout.txt stderr.txt
(base) ztmy@ubuntu:~/Epi1/e02$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt
(base) ztmy@ubuntu:~/Epi1/e02$ cat stdout.txt stderr.txt
/tmp:
config-err-enMqQz
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-colord.service-NNHt9C
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-fwupd.service-ApJrE3
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-rtkit-daemon.service-Pe5kG0
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-systemd-timesyncd.service-PvoHue
unity_support_test.0
VMwareDnD
vmware-root
ls: cannot access '/nonexistent': No such file or directory
(base) ztmy@ubuntu:~/Epi1/e02$ touch output.txt
(base) ztmy@ubuntu:~/Epi1/e02$ ls /nonexistent /tmp > output.txt 2>&1
(base) ztmy@ubuntu:~/Epi1/e02$ cat output.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
config-err-enMqQz
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-colord.service-NNHt9C
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-fwupd.service-ApJrE3
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-rtkit-daemon.service-Pe5kG0
systemd-private-be3dffccd7eb4f068ecbad2fa2c769fd-systemd-timesyncd.service-PvoHue
unity_support_test.0
VMwareDnD
vmware-root
(base) ztmy@ubuntu:~/Epi1/e02$

```

五、解题感悟

1. 了解了 Shell 中 stdin、stdout 和 stderr 三种标准流及其文件描述符。
2. 学会了使用 '>' 和 '2>' 分别重定向标准输出和标准错误。
3. 理解了 '2>&1' 的作用，掌握了将标准输出和错误输出合并到同一文件的方法。

练习 3：条件执行与退出状态控制

一、主题

Shell 基础与文件系统

二、题目要求

\$? 保存上一条命令的退出状态（0 表示成功）。&& 仅在前一条成功时执行后一条；|| 仅在前一条失败时执行后一条。写一个一行命令：仅当 /tmp/mydir 不存在时才创建它。

三、源程序代码

```
test -d /tmp/mydir || mkdir -p /tmp/mydir
```

四、程序运行结果

```
ztmy@ubuntu: ~/Epi1/e03
(base) ztmy@ubuntu:~$ cd ~/Epi1/e03
(base) ztmy@ubuntu:~/Epi1/e03$ test -d /tmp/mydir || mkdir -p /tmp/mydir
(base) ztmy@ubuntu:~/Epi1/e03$ ls -d /tmp/mydir
/tmp/mydir
(base) ztmy@ubuntu:~/Epi1/e03$
```

五、解题感悟

1. 学会了使用 `||` 根据上一条命令的执行结果决定是否执行后续命令。
2. 进一步理解了 `'/tmp'`、`'~'` 和 `'./'` 的区别，能够区分绝对路径和相对路径。
3. 通过实际操作发现，Shell 命令比较简单，但路径写错或者符号使用错误就容易导致命令执行失败。

练习 4：命令替换与日期格式化

一、主题

Shell 基础与文件系统

二、题目要求

写一条命令，把文件复制为带当天日期的备份文件名（例如 `notes.txt` → `notes_2026-01-12.txt`）。（提示：`$(date +%Y-%m-%d)`）

三、源程序代码

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

四、程序运行结果

```
ztmy@ubuntu: ~/Epi1/e04
(base) ztmy@ubuntu:~$ cd ~/Epi1/e04
(base) ztmy@ubuntu:~/Epi1/e04$ touch notes.txt
(base) ztmy@ubuntu:~/Epi1/e04$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
(base) ztmy@ubuntu:~/Epi1/e04$ ls
notes_2026-08-30.txt  notes.txt
(base) ztmy@ubuntu:~/Epi1/e04$
```

五、解题感悟

1. 学会了使用 ‘\$(...)’ 进行命令替换，将 ‘date’ 命令的结果直接用于文件名。
2. 进一步理解了 ‘%Y’、‘%m’、‘%d’ 等日期格式符号的含义，可以按照指定格式获取当前日期。
3. 通过实际操作掌握了利用当前日期生成备份文件名的方法，也认识到命令替换在 Shell 中可以提高命令使用的灵活性。

练习 5:

一、主题

Shell 管道与文本处理

二、题目要求

使用管道找出你「home 目录」中最常见的 5 种文件扩展名。（提示：组合 find、grep / sed / awk、sort、uniq -c 以及 head）

三、源程序代码

```
find ~ -type f | sed 's/.*\./' | grep -v '/' | sort | uniq -c | sort -nr | head -5
```

四、程序运行结果

```
ztmy@ubuntu: ~  
(base) ztmy@ubuntu:~$ find ~ -type f | sed 's/.*\.//' | grep -v '/' | sort | uni  
q -c | sort -nr | head -5  
find: '/home/ztmy/.dbus': Permission denied  
29654 py  
27376 pyc  
23949 h  
5123 cmake  
2809 json  
(base) ztmy@ubuntu:~$
```

五、解题感悟

1. 学会了使用 ‘find’ 查找文件，并通过管道把多个命令连接起来完成文件扩展名统计。
2. 进一步理解了 ‘sed’、‘sort’、‘uniq -c’ 和 ‘head’ 的作用，特别是学会了用 ‘sed’ 提取文件扩展名。
3. 通过这道题发现，多个简单命令组合起来也能完成比较复杂的任务，对管道的使用更加熟悉了。

练习 6：使用 find、xargs 统计 Shell 文件行数

一、主题

Shell 管道与文本处理

二、题目要求

- 1.xargs 会把 stdin 的每一行转换为命令参数。
2. 结合 find 和 xargs（不要用 find -exec），找出目录中所有.sh 文件，并用 wc -l 统计每个文件行数。
3. 加分项：正确处理文件名中的空格。（提示：-print0 和 -0）

三、源程序代码

```
find .. -type f -name "*.sh" -print0 | xargs -0 -n 1 wc -l
```

四、程序运行结果

```
ztmy@ubuntu: ~/Epi1/e06
(base) ztmy@ubuntu:~$ cd ~/Epi1/e06
(base) ztmy@ubuntu:~/Epi1/e06$ find .. -type f -name "*.sh" -print0 | xargs -0 -
n 1 wc -l
7 ../e01/check.sh
38 ../q02/analyze.sh
(base) ztmy@ubuntu:~/Epi1/e06$
```

五、解题感悟

1. 学会了使用 ‘find’ 查找指定类型的文件，并通过管道将结果传给 ‘xargs’ 继续处理。
2. 进一步理解了 ‘-print0’ 和 ‘-0’ 的作用，能够正确处理文件名中包含空格的情况。
3. 通过这道题熟悉了 ‘wc -l’ 的用法，也更加理解了多个 Shell 命令通过管道组合使用的方法。

练习 7：使用 git stash 暂存未提交的修改

一、主题

Version Control and Git

二、题目要求

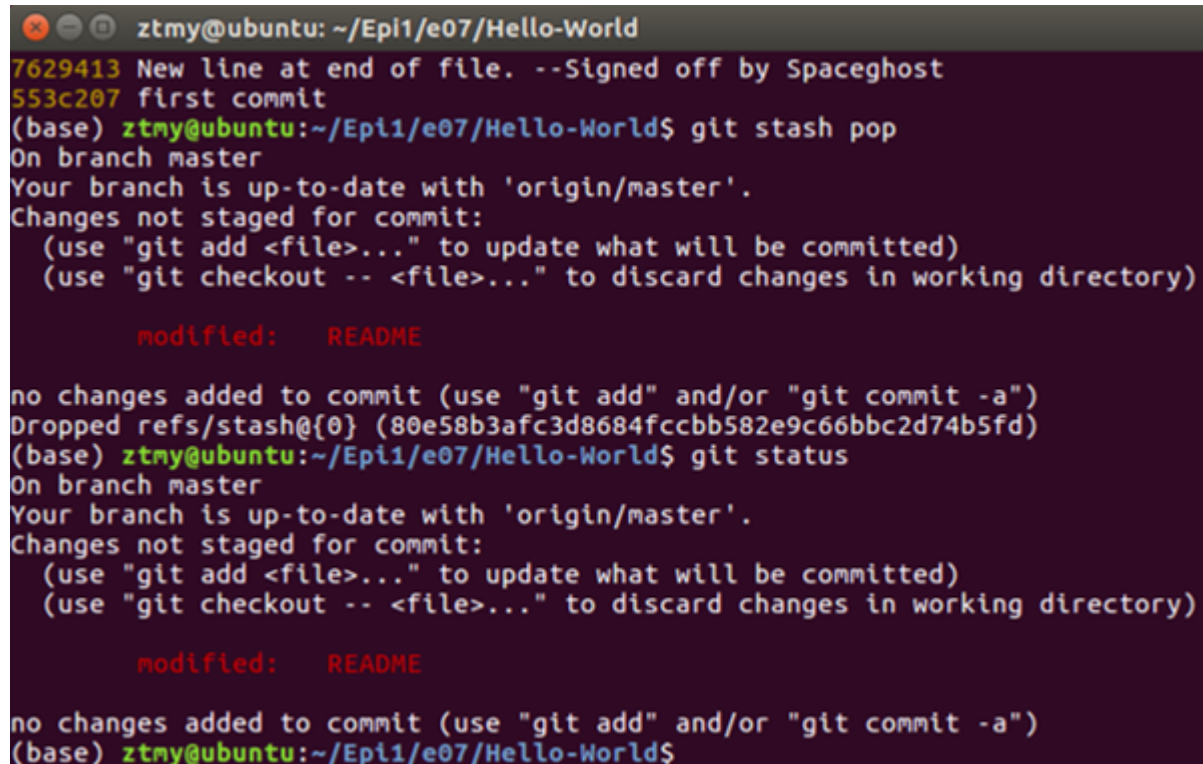
1. Clone 一个 GitHub 仓库，修改其中一个已有文件，然后执行 git stash，观察工作区变化。
2. 运行 git log --all --oneline 查看历史。
3. 再运行 git stash pop 恢复修改，并说明 git stash 在什么场景下有用。

三、源程序代码

```
cd ~/Epi1
git clone https://github.com/octocat/Hello-World.git
cd Hello-World
ls
echo "test change" » README
git status
git stash
```

```
git status
git log --all --oneline
git stash pop
git status
```

四、程序运行结果



```
ztmy@ubuntu: ~/Epi1/e07/Hello-World
7629413 New line at end of file. --Signed off by Spaceghost
553c207 first commit
(base) ztmy@ubuntu:~/Epi1/e07/Hello-World$ git stash pop
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (80e58b3afc3d8684fccbb582e9c66bbc2d74b5fd)
(base) ztmy@ubuntu:~/Epi1/e07/Hello-World$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
(base) ztmy@ubuntu:~/Epi1/e07/Hello-World$
```

五、解题感悟

1. 学会了使用 ‘git stash’ 临时保存没有提交的修改，并通过 ‘git stash pop’ 将修改恢复回来。
2. 通过实际操作了解了 ‘git stash’ 和 ‘git commit’ 的区别，知道 stash 保存的是临时修改，不会直接形成普通提交记录。
3. 通过这道题认识到，在需要临时切换分支或者处理其他任务时，可以使用 ‘git stash’ 保存当前工作，避免未完成的修改影响后续操作。

练习 8：使用 git config 创建命令别名

一、主题

Version Control and Git

二、题目要求

1. 和许多命令行工具一样，Git 也提供了一个配置文件（也叫 dotfile，隐藏配置文件），名为 ~/.gitconfig。
2. 请在 ~/.gitconfig 中创建一个别名，使得运行 git graph 时，可以得到 git log --all --graph --decorate --oneline 的输出。
3. 你可以直接编辑 ~/.gitconfig 文件，也可以使用 git config 命令来添加别名。

三、源程序代码

```
git config --global alias.graph 'log --all --graph --decorate --oneline'
```

四、程序运行结果

```
ztmy@ubuntu: ~/Epi1/e08
(base) ztmy@ubuntu:~$ cd ~/Epi1/e08
(base) ztmy@ubuntu:~/Epi1/e08$ git config --global alias.graph 'log --all --graph --decorate --oneline'
(base) ztmy@ubuntu:~/Epi1/e08$ git graph
* 4979287 (HEAD -> main, origin/main) add e07
* ca0c490 add e06
* 6382f92 add e04
* beda63b add e03
* a243cc7 add e02
* 4bf7c5d add e01
* c907984 add q04
* 2034bc3 add q03
* 2d9113d add q02
* 2b7bab1 add q01
(base) ztmy@ubuntu:~/Epi1/e08$
```

五、解题感悟

1. 学会了使用 ‘git config’ 配置 Git 别名，可以用简单的命令代替较长的命令。
2. 进一步了解了 ‘~/.gitconfig’ 文件的作用，知道它可以保存当前用户的 Git 配置。
3. 通过这道题认识到给常用命令设置别名可以减少重复输入，提高使用 Git 的效率。

练习 9：LaTeX 表格的创建与编译

一、主题

LaTeX 文档编辑

二、题目要求

Item	Quantity	Price(\$)
Nails	500	0.34
Wooden boards	100	4.00
Bricks	240	11.50

City	Year		
	2006	2007	2008
London	45789	46551	51298
Berlin	34549	32543	29870
Paris	49835	51009	51970

尝试画出下列表格：

三、源程序代码

```
cd ~/Epi1/e09
```

```
touch table.tex
```

```
vim table.tex
```

```
latexmk -pdf table.tex
```

```
xdg-open table.pdf
```

table.tex

```
\documentclass{article}
```

```
\begin{document}
```

```
\begin{tabular}{l|r|r}
```

```
\hline
```

```
Item & Quantity & Price(\$) \\\
```

```
\hline
```

```
Nails & 500 & 0.34 \\\
```

```
Wooden boards & 100 & 4.00 \\\
```

```
Bricks & 240 & 11.50 \\\
```

```
\end{tabular}
```

```
\vspace{1cm}
```

```
\begin{tabular}{l|r|r|r}
```

```
\hline
```

```
City & \multicolumn{3}{c}{Year} \\\
```

```
\cline{2-4}
```

```
& 2006 & 2007 & 2008 \\\
```

```
\hline
```

```
London & 45789 & 46551 & 51298 \\\
```

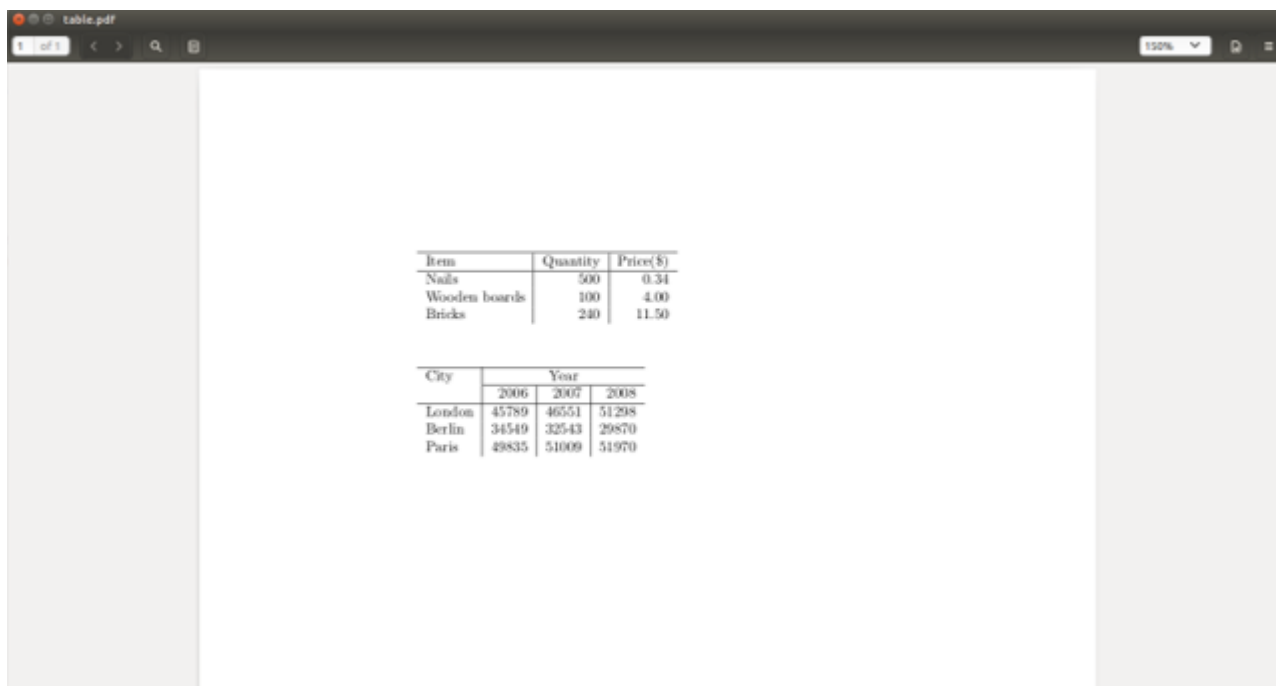
```
Berlin & 34549 & 32543 & 29870 \\\
```

```
Paris & 49835 & 51009 & 51970 \\\
```

```
\end{tabular}
```

```
\end{document}
```

四、程序运行结果



五、解题感悟

1. 学会了使用 ‘tabular’ 环境创建 LaTeX 表格，并了解了 ‘&’、‘\\’ 和 ‘\hline’ 的基本用法。
2. 通过编译过程中出现的报错，发现 LaTeX 对命令拼写要求比较严格，一个字母写错也可能产生多个错误。
3. 了解了 ‘pdflatex’ 和 ‘latexmk’ 的区别，知道简单文档可以直接使用 ‘pdflatex’ 编译，而复杂文档可以使用 ‘latexmk’ 自动完成多次编译。

练习 10：LaTeX 图片的插入与排版

一、主题

LaTeX 文档编辑

二、题目要求

1. 在你文档的前导命令中添加 `\usepackage{graphicx}`。
2. `\rightarrow` 找到一张图片，放置在你的 LaTeX course 文件夹下。
3. 在你想要添加图片的地方输入以下内容：

```
\begin{figure}[h!]
\centering
\includegraphics[width=1\textwidth]{ImageFilename}
\caption{My test image}
\end{figure}
```

三、源程序代码

```
cd ~/Epi1/e10
touch image.tex
vim image.tex
```

```
latexmk -pdf image.tex
```

```
xdg-open image.pdf
```

image.tex

```
\documentclass{article}
```

```
\usepackage{graphicx}
```

```
\begin{document}
```

This is my terminal image:

```
\begin{figure}[h!]
```

```
\centering
```

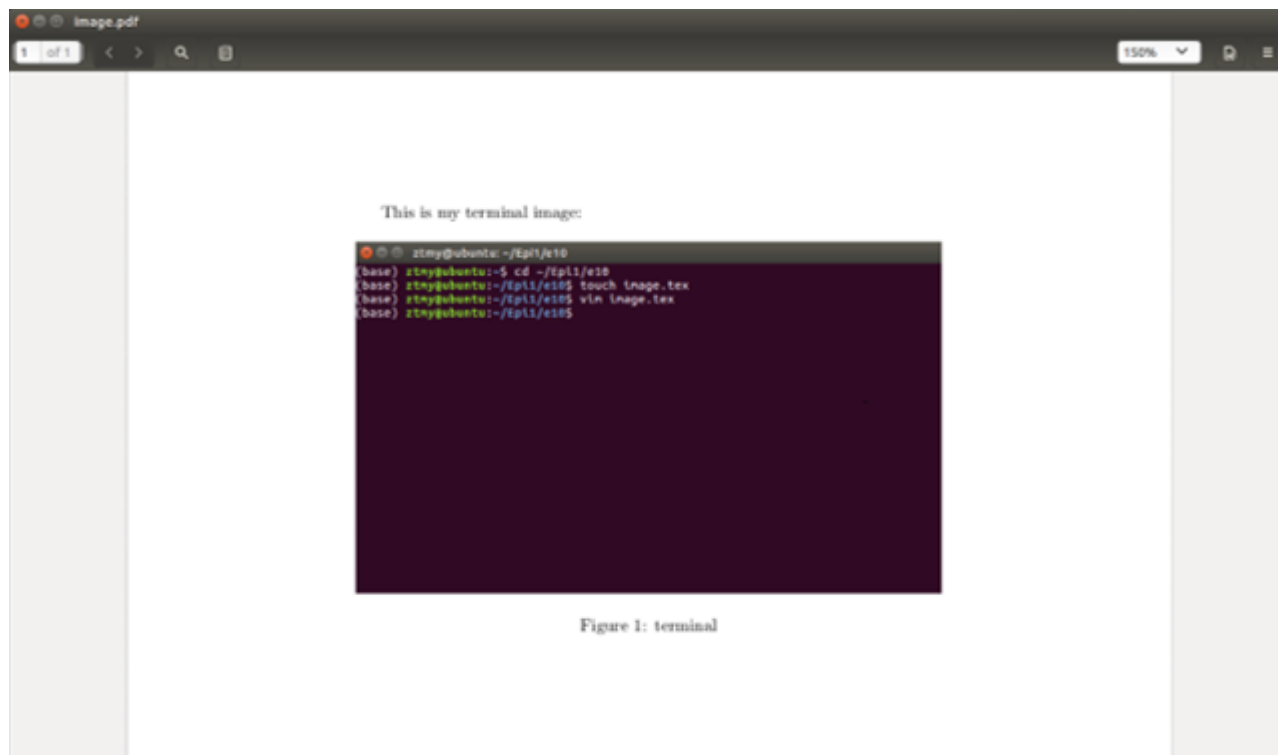
```
\includegraphics[width=1\textwidth]{terminal}
```

```
\caption{terminal}
```

```
\end{figure}
```

```
\end{document}
```

四、程序运行结果



五、解题感悟

1. 学会了使用 ‘graphicx’ 宏包和 ‘\includegraphics’ 命令在 LaTeX 中插入图片。
2. 了解了 ‘figure’、‘\centering’ 和 ‘\caption’ 的基本作用，可以对插入的图片进行简单的排版和添加标题。
3. 通过实际编译发现，图片文件的位置和文件名需要与 LaTeX 文件对应，否则编译时容易出现找不到图片的问题。

练习 11：LaTeX 数学公式的编写

一、主题

LaTeX 文档编辑

二、题目要求

撰写代码来生成下列公式：

$$e = mc^2 \quad (1)$$

$$\pi = \frac{c}{d} \quad (2)$$

$$\frac{d}{dx}e^x = e^x \quad (3)$$

$$\frac{d}{dx} \int_0^\infty f(s)ds = f(x) \quad (4)$$

$$f(x) = \sum_i 0^\infty \frac{f^{(i)}(0)}{i!} x^i \quad (5)$$

$$x = \sqrt{\frac{x_i}{z}} y \quad (6)$$

三、源程序代码

```
cd ~/Epi1/e11
```

```
touch formula.tex
```

```
vim formula.tex
```

```
latexmk -pdf formula.tex
```

```
xdg-open formula.pdf
```

formula.tex

```
\documentclass{article}
```

```
\begin{document}
```

```
\begin{equation}
```

```
e = mc^2
```

```
\end{equation}
```

```
\begin{equation}
```

```
\pi = \frac{c}{d}
```

```
\end{equation}
```

```
\begin{equation}
```

```
\frac{d}{dx}e^x = e^x
```

```
\end{equation}
```

```
\begin{equation}
```

```
\frac{d}{dx} \int_0^\infty f(s)ds = f(x)
```

```
\end{equation}
```

```
\begin{equation}
```

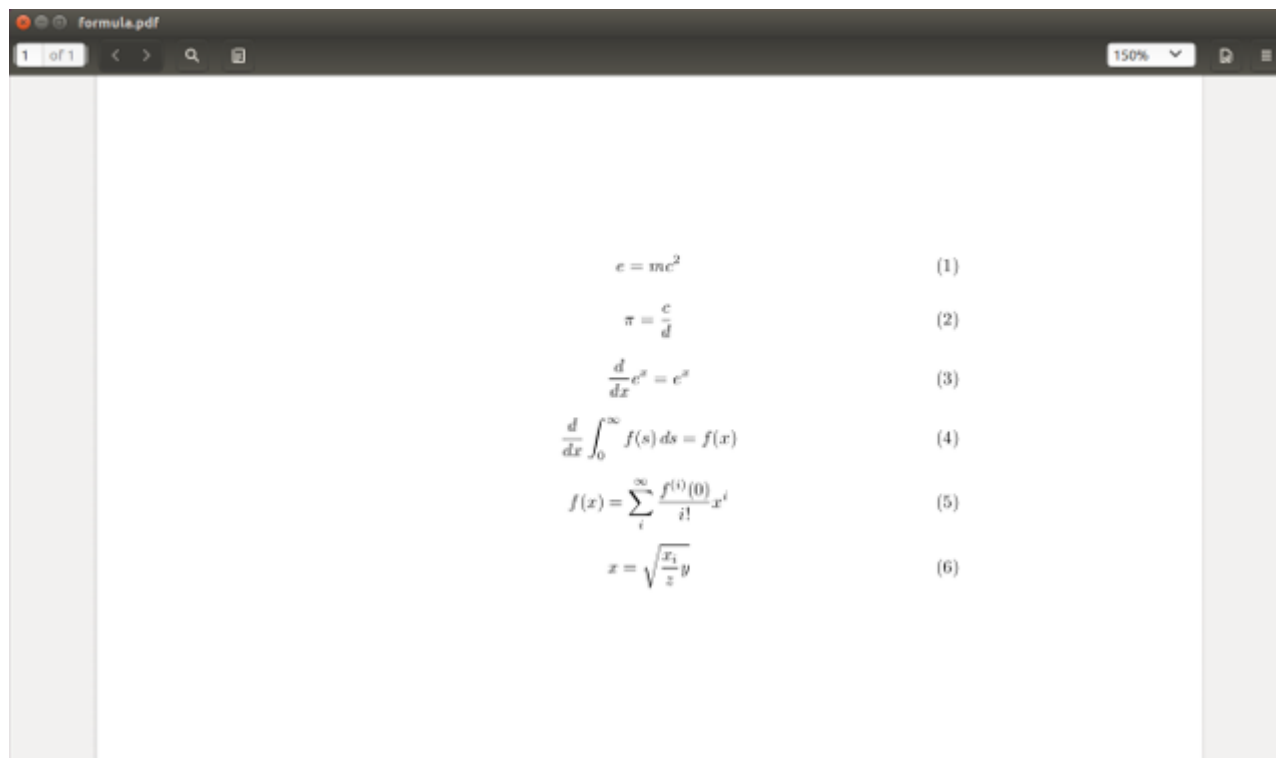
```
f(x) = \sum_i 0^\infty \frac{f^{(i)}(0)}{i!} x^i
```

```
\end{equation}
```

```
\begin{equation}
```

```
x = \sqrt{\frac{x_i}{z}y}
\end{equation}
\end{document}
```

四、程序运行结果



五、解题感悟

1. 学会了使用 ‘equation’ 环境编写数学公式，并了解了公式编号的生成方式。
2. 熟悉了 ‘\frac’、‘\sqrt’、‘\sum’ 以及上下标等常用数学公式命令。
3. 通过实际编写不同形式的公式，对 LaTeX 数学公式的基本语法有了更深入的了解。